

DOCUMENTATION REPORT

SCOA031 | GROUP 8

1. 240074124 KEKANA THAPELO SELOGADI
2. 202253761 MUKWEVHO TSHIMANGADZO
3. 202204262 RAKGETSE PHOOKO IRVIN
4. 202351423 MAMPHWE VHUHWAVHO LUCIA
5. 240026528 MAAHLAMELA MOLOTO SURPRISE
6. 202308784 BADA SHUMANI TECTAR
7. 240014384 MAKHUELE MASANA
8. 202409799 SAMBO NKOSINGIPHILE LUSANDA
9. 202234828 MASIKHWA KHUTHADZO LEWIS
10. 202219584 MNGUNI THOKOZANI
11. 240021616 LUSUNZI RINAE

1. System Design

Overview

The system is built using the AdventureWorks2022 database and follows a structured database architecture. It uses stored procedures, triggers, indexing, and transactions to ensure efficiency, integrity, and reliability.

Stored Procedures

- AddNewCustomer:
Inserts customer data into multiple related tables using transactions and error handling.
- UpdateProductPrice:
Updates product prices with validation.
- Deleting/archiving InactiveCustomers
Deleting/Archiving inactive customers based on last purchase
- MonthlySalesReport:
Aggregates sales per month.
- Top10BestSellingProducts:
Returns top-selling products.
- EmployeePerformanceHistory:
Tracks employee order handling.
- Monitoring procedures:
Detect long queries, database size, and index fragmentation.

2. Security Model

The system implements Role-Based Access Control (RBAC).

Roles and Permissions

-SalesRole:

SELECT, INSERT, UPDATE on Sales schema; denied access to HR.

-HRRole:

SELECT, INSERT, UPDATE on HR schema; denied access to Sales.

- DBA_Role:

Full database access.

Users

SalesUser → SalesRole

HRUser → HRRole

DBAUser → DBA_Role

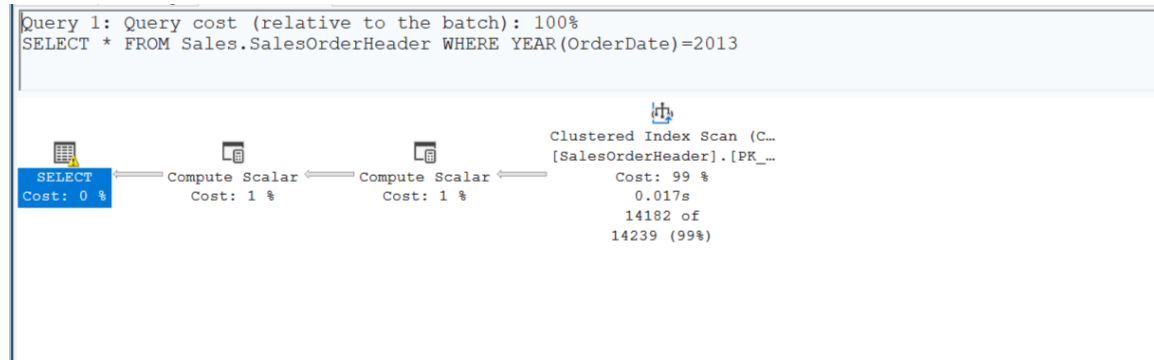
3. Performance Improvements

Before Optimization

Queries were slow due to use of functions like YEAR(OrderDate) .

Example:

WHERE YEAR(OrderDate)=2014

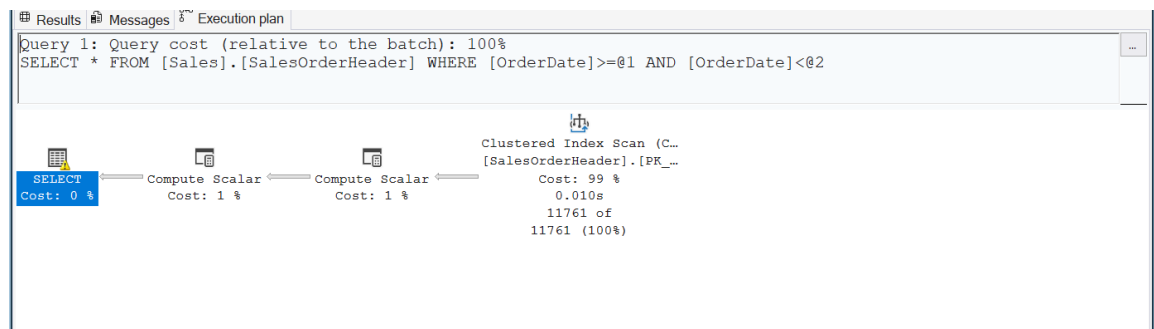


After Optimization

Queries rewritten using date ranges.

Example:

WHERE OrderDate BETWEEN '2014-01-01' AND '2014-12-31'



Created Index

CREATE INDEX IN_ORDERQTY
ON Sales.SalesOrderDetail(OrderQty)

Before index:

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query with line numbers 1 through 18. The query text is: `SET STATISTICS TIME ON` (line 1), `SELECT * FROM Sales.SalesOrderDetail` (line 2), and `WHERE OrderQty=1` (line 3). The bottom pane shows the execution results. It includes a progress bar at 91% and a status message 'No issues found'. The 'Results' tab is active, showing the following output:
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 4 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(74954 rows affected)

SQL Server Execution Times:
CPU time = 62 ms, elapsed time = 1275 ms.

Completion time: 2026-05-01T11:25:28.7978444+02:00
|

After index:

SQLQuery4....mohal (54))* PART B.sql -...E\mohal (55))

1

2

3

4

5

6

7

8

SET STATISTICS TIME ON

CREATE INDEX IN_ORDERQTY

ON Sales.SalesOrderDetail(OrderQty)

SELECT * FROM Sales.SalesOrderDetail

WHERE OrderQty=1

91 %

✓ No issues found

Results

Messages

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 4 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(74954 rows affected)

SQL Server Execution Times:
CPU time = 46 ms, elapsed time = 1189 ms.

Completion time: 2026-05-01T11:26:04.6251281+02:00

4. Trigger & Alert Design

Audit Table

Stores table name, action type, user, timestamp, and description.

Triggers

Triggers are created on Customer and Product tables for INSERT, UPDATE, DELETE operations.

Logging Process

1. User performs action.
2. Trigger fires automatically.
3. Data is logged into AuditTable with user and timestamp.

Alerts

System logs all activities and can send alerts using Database Mail and SQL Server Agent.

5. Backup Strategy

Backup Types

Full Backup: Weekly

Differential Backup: Daily

Transaction Log Backup: Every 15–30 minutes

Recovery Plan

1. Restore full backup.
2. Restore differential backup.
3. Restore transaction logs for point-in-time recovery.